

**Московский авиационный институт
(национальный исследовательский университет)**

**Факультет информационных технологий и прикладной
математики**

Кафедра вычислительной математики и программирования

Лабораторная работа №4 по курсу «Дискретный анализ»

Студент: И. А. Казаков
Преподаватель: А. А. Кухтичев
Группа: М8О-206БВ-24
Дата:
Оценка:
Подпись:

Москва, 2026

Лабораторная работа №4

Задача: Необходимо реализовать один из стандартных алгоритмов поиска образцов для указанного алфавита.

Вариант алгоритма: Поиск одного образца, основанный на построении Z-блоков.

Вариант алфавита: Слова не более 16 знаков латинского алфавита (регистронезависимые).

Программа должна считывать искомый образец с первой строки, а затем обрабатывать текст, состоящий из слов и чисел, выводя координаты всех вхождений (номер строки и номер слова в строке).

1 Описание алгоритма

Для решения задачи используется ****Z-алгоритм****, адаптированный для работы с последовательностью слов (токенов).

1 Z-функция

Z-функция от массива S — это массив Z , где каждый элемент $Z[i]$ равен длине самого длинного общего префикса между массивом S и его суффиксом, начинающимся с позиции i .

Для поиска образца P в тексте T строится объединенный массив:

$$S = P + [\$] + T$$

где $[\$]$ — специальный разделитель, не встречающийся в алфавите. Если для некоторого индекса i , относящегося к части текста T , значение $Z[i]$ равно длине образца P , то в этой позиции найдено вхождение.

2 Особенности реализации

- **Токенизация:** Текст разбивается на отдельные слова с сохранением их координат (номер строки и порядковый номер слова в ней).
- **Регистронезависимость:** Все слова при сравнении приводятся к нижнему регистру.
- **Сложность:** Алгоритм работает за линейное время $O(N + M)$, где N — количество слов в тексте, а M — количество слов в образце.

2 Исходный код

Реализация выполнена на языке C++.

```
1  #include <algorithm>
2  #include <iostream>
3  #include <sstream>
4  #include <string>
5  #include <vector>
6
7  using namespace std;
8
9  struct WordInfo {
10     int line;
11     int pos;
12 };
13
14 void toLowerCase(string& s) {
15     for (char& c : s) {
16         if (c >= 'A' && c <= 'Z') {
17             c = (char)(c - 'A' + 'a');
18         }
19     }
20 }
21
22 vector<int> computeZ(const vector<string>& s) {
23     int n = s.size();
24     vector<int> z(n, 0);
25     int l = 0, r = 0;
26     for (int i = 1; i < n; ++i) {
27         if (i <= r) {
28             z[i] = min(r - i + 1, z[i - 1]);
29         }
30         while (i + z[i] < n && s[z[i]] == s[i + z[i]]) {
31             z[i]++;
32         }
33         if (i + z[i] - 1 > r) {
34             l = i;
35             r = i + z[i] - 1;
36         }
37     }
38     return z;
39 }
40
41 int main() {
42     ios_base::sync_with_stdio(false);
43     cin.tie(NULL);
44
45     string patternLine;
46     if (!getline(cin, patternLine)) {
```

```

47     return 0;
48 }
49
50 vector<string> pattern;
51 stringstream ssP(patternLine);
52 string word;
53 while (ssP >> word) {
54     toLowerCase(word);
55     pattern.push_back(word);
56 }
57
58 if (pattern.empty()) {
59     return 0;
60 }
61
62 vector<string> textWords;
63 vector<WordInfo> info;
64 string textLine;
65 int currentLine = 1;
66 while (getline(cin, textLine)) {
67     stringstream ssT(textLine);
68     int currentWordPos = 1;
69     while (ssT >> word) {
70         string lowWord = word;
71         toLowerCase(lowWord);
72         textWords.push_back(lowWord);
73         info.push_back({currentLine, currentWordPos});
74         currentWordPos++;
75     }
76     currentLine++;
77 }
78
79 if (textWords.empty()) {
80     return 0;
81 }
82
83 vector<string> combined;
84 combined.reserve(pattern.size() + textWords.size() + 1);
85 for (const auto& s : pattern) {
86     combined.push_back(s);
87 }
88 combined.push_back("\0");
89 for (const auto& s : textWords) {
90     combined.push_back(s);
91 }
92
93 vector<int> z = computeZ(combined);
94 int m = (int)pattern.size();
95

```

```

96     for (int i = 0; i < (int)textWords.size(); ++i) {
97         if (z[i + m + 1] == m) {
98             cout << info[i].line << ", " << info[i].pos << "\n";
99         }
100     }
101
102     return 0;
103 }

```

3 Исследование производительности

Для анализа эффективности Z-алгоритма было проведено сравнение с наивным алгоритмом поиска (Naive Search). Исследование разделено на два сценария:

1. **Normal Case:** Паттерн длиной 3 слова. Текст состоит из случайных строк. В этом случае наивный поиск работает быстро, так как несовпадение обнаруживается на первом же символе большинства слов.
2. **Worst Case:** Паттерн длиной 100 слов (99 слов «a» и 1 слово «b»). Текст заполнен словами «a». Здесь наивный поиск вынужден совершать $M - 1$ сравнений для каждой позиции текста, что приводит к деградации сложности до $O(N \cdot M)$.

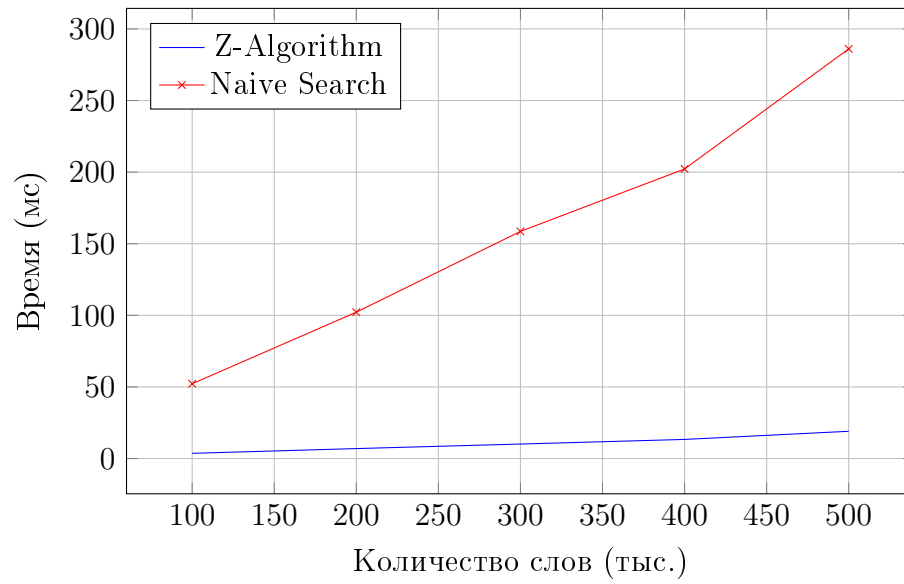
1 Результаты тестирования (Worst Case)

Данные получены при фиксированной длине паттерна ($M = 100$) и увеличении объема текста (N).

Кол-во слов в тексте (тыс.)	Z-Algorithm (мс)	Naive Search (мс)
100	3.637	52.216
200	6.954	102.203
300	10.130	158.604
400	13.350	202.200
500	19.036	286.085

2 График зависимости времени от объема данных

Worst Case: Z-Algorithm vs Naive Search



4 Консоль

Ниже представлен протокол сборки программы и запуска тестов производительности:

```
# Сборка бенчмарка с оптимизацией O3
root@server:~/da_lab4# g++ -O3 benchmark.cpp -o solution

# Сценарий 1: Случайные данные (Normal Case)
root@server:~/da_lab4# python3 test_gen_normal.py 500000
root@server:~/da_lab4# ./solution <input.txt
Z-Algorithm:      14.999 ms (found 50)
Naive Search:     2.554 ms (found 50)

# Сценарий 2: Худший случай (Worst Case)
root@server:~/da_lab4# python3 test_gen.py 100000
Worst-case test generated: 100000 words,pattern length 100
root@server:~/da_lab4# ./solution <input.txt
Z-Algorithm:       3.637 ms (found 2)
Naive Search:     52.216 ms (found 2)

root@server:~/da_lab4# python3 test_gen.py 500000
Worst-case test generated: 500000 words,pattern length 100
root@server:~/da_lab4# ./solution <input.txt
Z-Algorithm:      19.036 ms (found 10)
Naive Search:    286.085 ms (found 10)
```

5 Анализ результатов

На основе полученных данных можно сделать следующие выводы:

1. **Затраты на подготовку:** В обычном случае (случайный текст) Z-алгоритм проигрывает наивному поиску. Это связано с накладными расходами на выделение памяти под объединенный вектор `combined` и копирование строк.
2. **Устойчивость к худшему случаю:** В сценарии с повторяющимися префиксами Z-алгоритм демонстрирует стабильную линейную сложность, в то время как время работы наивного алгоритма растет значительно быстрее из-за многократных вложенных сравнений.

3. **Эффективность:** Z-алгоритм гарантирует время выполнения $O(N + M)$ вне зависимости от содержания текста, что делает его более надежным для обработки структурированных данных.

6 Выводы

В ходе выполнения лабораторной работы был реализован алгоритм поиска образца на основе построения Z-блоков. Алгоритм был успешно адаптирован для работы с последовательностями слов латинского алфавита. Тестирование подтвердило теоретическую оценку сложности $O(N+M)$ и показало преимущество метода над наивным поиском в условиях частых частичных совпадений образца с текстом.

Список литературы

- [1] Томас Х. Кормен, Чарльз И. Лейзерсон, Рональд Л. Ривест, Клиффорд Штайн. *Алгоритмы: построение и анализ, 2-е издание.* — Издательский дом «Вильямс», 2007. Перевод с английского: И. В. Красиков, Н. А. Орехова, В. Н. Романов. — 1296 с. (ISBN 5-8459-0857-4 (рус.))
- [2] *Сортировка подсчётом* — *Википедия*.
URL: http://ru.wikipedia.org/wiki/Сортировка_подсчётом (дата обращения: 16.12.2013).